



North South University

Department of Electrical and Computer Engineering

CSE327 — Software Engineering

Project Report

SplitWise+

Instructor: Dr. Md Adnan Arefeen [AFE]

Submitted by

Jannat-a-habib-baishakhi (2131069642)

Sakila Anjum (2131241642)

1. Introduction

Managing shared money is difficult when expenses, conversations, decisions, and payment records are distributed across different tools. In a trip, shared apartment, student group, or club, members may know who paid but still be uncertain about participants, individual shares, outstanding balances, or the next required transfer. Spending decisions also commonly take place in a separate chat, which causes duplicated information and weak financial traceability.

SplitWise+ addresses this problem through a single collaborative workspace. The platform is a BDT-first financial application for groups that share expenses, plans, responsibilities, and decisions. It combines a shared expense ledger with settlement workflows, group planning, real-time communication, profile management, and account security.

The main objective was to develop a functional, backend-connected platform that makes shared money transparent and actionable. The project specifically aimed to maintain an accurate shared ledger, simplify settlement, support collaborative planning, and keep financial conversations connected to the records they discuss.

Problem	SplitWise+ solution
Unclear shared expenses	Store amount, payer, participants, category, notes, and split information together.
Difficult payment follow-up	Show balance breakdowns, optimized settlement actions, and payment status.
Planning separated from finance	Provide budgets, polls, events, RSVPs, and recurring expenses inside the group workspace.
Unreliable demo state	Use authenticated REST APIs, persistent models, real empty states, and backend-authoritative totals.

2. Implemented Features

2.1 Authentication, groups, and invitations

Users can create accounts and sign in through the Django JWT API. Authenticated sessions persist access and refresh tokens in browser storage, and users can sign out through the account menu or Settings. New users are not automatically joined to a group. Instead, membership is created explicitly through group creation, username-based invitations, or invitation acceptance.

Groups provide the main authorization boundary. Members access only the expenses, balances, plans, messages, and shared files associated with their groups. The invitation flow searches real database users, sends an invitation to a selected username, and supports acceptance or decline from the recipient's account area. Backend membership checks protect group-scoped data and mutations.

2.2 Dashboard and expense management

The personal Dashboard aggregates backend data across the current user's groups. It shows total shared spend, recorded payments, participant share, and the amount waiting to settle. Two charts visualize Shared spend by group and Money position, comparing group totals and the user's paid, share, to-pay, and to-receive values.

The Dashboard is not populated with fabricated chart values. It refreshes from the authenticated backend endpoint after relevant mutations and at regular intervals. If no expenses exist, the charts still show axes, gridlines, a zero baseline, labels, and guidance for recording the first transaction. This makes an empty result understandable instead of presenting a blank card.

The expense ledger supports creating, updating, searching, filtering, and deleting shared transactions. Users can record a BDT amount, payer, participants, category, date, notes, and receipt state. The split flow supports equal, exact, and percentage-based allocation. Participant-share records are persisted by the backend and reused by Dashboard and settlement calculations.

2.3 Balances and settlement

The settlement workflow converts expense records into actionable payment instructions. It identifies members who owe money and members who should receive money, while reducing unnecessary transfers where possible. Settlement requests can be reviewed and payment status can be confirmed. The Dashboard exposes the outstanding amount so that a balance remains visible after an expense is recorded.

2.4 Planning, budgets, and recurring expenses

The Plan page connects future decisions to financial commitments. Its shared calendar supports group event creation and RSVP-oriented coordination. The decisions area supports publishing polls and collecting votes. The page also exposes recurring-expense scheduling for commitments that repeat over time.

Budgets allow a group to set category-based spending limits and compare actual expenses against a plan. Recurring Expenses support weekly, monthly, or quarterly shared commitments such as rent, subscriptions, or utilities. The pages use truthful empty states: when no budget, event, poll, or recurring expense exists, the interface explains what the user can create and keeps the creation controls available.

2.5 Real-time messaging

The Messages workspace follows a Messenger and WhatsApp-style interaction model. It includes group threads, private-message entry points, member profile drawers, reactions, reply context, delivery state, typing indicators, read-state hooks, chat themes, and shared-media metadata. Users can attach documents and images.

2.6 Settings, profiles, and account security

Settings consolidates profile and security controls. Users can edit status and biography, upload a profile picture through a multipart request, choose dark or light theme, and view account identity details. Clicking the avatar or name opens a standard account menu with theme controls, Settings, and sign out; Settings is intentionally not kept as a separate permanent sidebar item.

The security section lists active sessions, identifies the current device, shows recent activity, and allows users to revoke unfamiliar sessions or sign out everywhere. A persistent activity log records sign-ins, profile changes, sign-outs, and session actions. Session-aware JWT (JSON Web Token) authentication rejects tokens associated with revoked sessions, so revocation is enforced at the API level.

3. Architecture and Technology Stack

SplitWise+ follows a layered full-stack architecture. The React frontend is responsible for the interface, client state, formatting, navigation, forms, charts, and WebSocket clients. The Django backend owns authentication, permissions, domain rules, database persistence, aggregation, media handling, and realtime event delivery.

Layer	Technologies	Main responsibility
Frontend	React 19, TypeScript, Vite, native CSS, Lucide React	Responsive workspace, forms, charts, chat, media controls, and Settings.
Backend	Django 5, Django REST Framework	Authenticated APIs, serializers, permissions, groups, expenses, settlements, plans, and profiles.
Data and media	SQLite, Django media storage	Persistent users, groups, finance records, sessions, activity, receipts, avatars, and attachments.
Version control	Git and GitHub	Source history, collaboration, backup, and release synchronization.

The backend is divided into domain-oriented applications. `apps/accounts` provides the authentication boundary, `apps/groups` provides group APIs, `apps/finance` contains expense and settlement surfaces, `apps/messaging` contains chat and realtime consumers, `apps/planning` contains planning services, and `apps/core` contains shared models, migrations, compatibility APIs, and the repeatable `seed_demo` command.

4. Project Progress

The project progressed through four major updates, moving from a visual prototype to a connected collaborative application.

Update	Progress completed
Update 1	Initially developed the frontend with React JS. The interface contained dummy data and was used to establish the initial screens, navigation, and visual direction.
Update 2	Developed the Django backend and redesigned the UI around the liquid-glass workspace style. Domain models, modular backend apps, BDT formatting, and persistence were introduced.
Update 3	Integrated the backend APIs with the frontend. Authentication, groups, invitations, expenses, settlements, Dashboard aggregation, planning, and profile operations became backend-connected and persistent.
Update 4	Implemented chat features, including group and private messaging, reactions, media controls, profile interactions, WebSocket-ready delivery, typing/read hooks, and chat themes.

The progression also led to additional account features. Profile-picture upload, theme persistence, activity history, and active-session management were added after the main chat integration. These features strengthened the product's security and made the account experience functional rather than decorative.

6. Conclusion

SplitWise+ demonstrates an incremental software-engineering process in which an initial React prototype was transformed into a modular, backend-connected product. The final system combines BDT-first expense tracking, participant splitting, settlements, group invitations, budgets, polls, events, recurring expenses, real-time messaging, rich media controls, profile management, activity history, and active-session security.

The central contribution of the project is the connection between financial records and the social context surrounding them. A group can understand what was spent, discuss why it was spent, decide what should happen next, and settle balances without leaving the same workspace. The separation between React presentation, Django domain logic, persistent data creates a clear technical foundation for future production hardening.

GitHub Repository: <https://github.com/jannatbaishakhi/CSE327-Project>